

AST-SELF-CHECK: CALIBRATED, TEST-OPTIONAL, HIERARCHICAL ERROR PROBABILITIES FOR SELF-DEBUGGING CODE LLMs

Anonymous authors

Paper under double-blind review

ABSTRACT

We study verification and localization of bugs in code produced by language models when gold unit tests are missing or incomplete. Existing self-debug pipelines rely on small tests or natural-language critiques that are often overconfident and not localized, and most calibration efforts estimate only global correctness [chen2023teaching, olausson2023self, xiong2023can]. We propose *AST-SelfCheck*, a drop-in verifier that is language-agnostic and test-optional. It mines conservative executable specifications into a portable DSL for property-based and metamorphic testing, synthesizes interfaces preserving code variants, compares structure and behavior at the AST level, fuses dynamic, static, symbolic, and based fault localization signals, and outputs calibrated hierarchical $P(\text{error})$ with conformal confidence intervals. A controller allocates compute and stops early when confidence thresholds are met, and optional patch intent calibrators and conformal prediction, we observe function level ROCAUC 0.723, average precision 0.520, accuracy around 0.70 0.73, expected calibration error 0.187, maximum calibration error 0.341, and Brier score 0.200 on 40 test samples. We discuss limitations, and outline a roadmap for line level calibration, coverage audits, and pass@k improvements under fixed compute.

1 INTRODUCTION

Large language models trained on code achieve strong performance on short problems but still fail frequently, particularly when prompts are ambiguous or when unit tests are sparse [austin2021program, chen2021evaluating]. Self-repair strategies that iterate critique and edit have emerged as a remedy, yet their efficacy is bottlenecked by the quality of small test suites, miss faults, and natural-language critique tends to be overconfident, uncalibrated, and poorly localized in predicting whether an entire program is correct—rather than localized to functions and lines. Few approaches provide aware allocation [kapoor2024large, shen2024hermometer]. These gaps matter for building reliable autonomous coding agents.

We address the verification bottleneck with *AST-SelfCheck*, a test-optional, multi-signal verifier that returns calibrated, hierarchical probabilities of error along with executable counterexamples and interpretable heatmaps. The core idea is to construct conservative executable oracles from documentation and code, generate interface-preserving program variants, compare them at the abstract syntax tree and behavioral levels, and fuse these signals with coverage spectra, static analysis, and selective symbolic reasoning under a compute budget. We then learn a meta-calibrator that outputs $P(\text{function buggy})$ and $P(\text{line buggy} \mid \text{function buggy})$, and report conformal confidence intervals to support early stopping and targeted repairs.

This problem is challenging. Gold tests are often missing or incomplete; mined properties may be noisy or vacuous; dynamic testing and symbolic tools are budget-limited; spectra can be unstable under flakiness; and calibration must respect structured dependencies among lines and functions. Prior verification and selection approaches typically assume tests, rely on generated tests, or operate at a holistic level [chen2022odet, miao2023selfcheck, wei2022chain, wang2022self]. Calibration efforts for LLMs have focused on global, not more sampling—limits per performance Olausson et al. (2023).

We formalize a drop-in interface `detect(prompt, code, budget)` that returns function- and line-level heatmaps, counterexamples, violated predicates, top signals, hierarchical probabilities with confidence

intervals, and optional patch intents. The system comprises: a conservative specification-mining DSL for property and metamorphic testing; regenerate-and-compare at the AST level to produce structural and behavioral divergence signals; hybrid verification combining dynamic, static, and symbolic engines under budget control; variant ensembles plus spectrum-based fault localization; a meta-calibrator with temperature scaling and Mondrian conformal prediction; a budget-aware controller that allocates compute by expected utility; and optional patch intents for minimal, interface-preserving repairs.

We implemented a Python prototype with PyTorch MLP meta-calibrators, temperature scaling, and conformal prediction. On 200 synthetic functions (86 buggy, 114 correct), a held-out 40-sample test set yields ROC AUC 0.723, average precision 0.520, accuracy about 0.70-0.73, ECE 0.187, MCE 0.341, and Brier 0.200. Per-function outputs include predicted P(error) and counts of failing tests. While these results are function-level and do not yet validate hierarchical line-level calibration, they demonstrate that a test-optional, multi-signal verifier can meaningfully discriminate buggy code and produce uncertainty estimates.

- **Calibrated, localized verifier:** A test-optional, language-agnostic verifier that outputs calibrated, hierarchical per-function and per-line P(error) with confidence intervals, plus executable counterexamples and variant-driven AST signals.
- **Portable specification DSL:** A portable specification-mining DSL that emits property-based and metamorphic tests without gold outputs, coupled with conservative reliability controls.
- **Meta-calibration pipeline:** A meta-calibration pipeline that fuses dynamic, static, symbolic, spectrum-based, and regenerate-and-compare features with temperature scaling and conformal prediction adapted to structured code partitions [kapoor₂₀₂₄*large*, shen₂₀₂₄*hermometer*]. **Budget-aware control** : *Abudget – awarecontrollerwithearlystoppingandoptionalpatchintentsforhigh – confidencefaulttypes,designedtoimprovepass@k at fixed compute* Olausson et al. (2023).
- **Prototype evidence:** A prototype evaluation showing promising function-level discrimination and moderate calibration, alongside a plan for line-level localization and pass@k improvements under compute parity.

Looking ahead, we will: (i) finalize hierarchical line-level calibration and conduct coverage audits with Nested-Mondrian partitions; (ii) expand to JavaScript/TypeScript and C++ using portable test harnesses; (iii) evaluate pass@k uplift in self-debug pipelines under compute parity; and (iv) compare against gold-only, fuzz-only, static-only, and NL-critique baselines to close the feedback bottleneck [chen₂₀₂₃*eaching*, olausson₂₀₂₃*self*, chen₂₀₂₂*odet*, author_{year}*lm*].

2 RELATED WORK

2.1 SELF-DEBUG AND REPAIR

Teaching LLMs to critique and edit their own code improves accuracy on several benchmarks, but the gains strongly depend on the quality of feedback and available repair budget Chen et al. (2023). When costs are accounted for, performance improvements can be modest or inconsistent, suggesting that self-repair is often bottlenecked by weak verifiers and uncalibrated feedback Olausson et al. (2023). AST-SelfCheck targets this bottleneck by replacing uncalibrated natural-language critique with calibrated, localized probabilities and executable counterexamples.

2.2 VERIFICATION AND SELECTION WITHOUT FULL TESTS

CodeT generates tests and uses dual-execution agreement to select among candidate programs Chen et al. (2022). In reasoning, self-consistency aggregates multiple sampled chains to select consistent answers [wang₂₀₂₂*self*, wei₂₀₂₂*chain*], and *SelfCheckverifiesstep – by – stepreasoningwithLLMsMiao et al. (2023).Theselinesofworkeitherrelyongeneratedtestsoroperateataholisticlevel. SelfCheckistest – optionalandproducescalibrated, hierarchicalprobabilitieswithconfidenceintervalsandper – linelocalizationtoguidetargetedrepair.*

2.3 CALIBRATION FOR LLMs

Prompting alone is generally insufficient for reliable calibration; small auxiliary probes with temperature scaling can yield better-calibrated uncertainties and generalize across tasks Kapoor et al. (2024). Thermometer introduces an auxiliary-model approach for universal calibration across diverse tasks Shen et al. (2024). Black-box verbalized confidence and response-consistency partially mitigate overconfidence but remain unreliable on challenging tasks Xiong et al. (2023). AST-SelfCheck adopts lightweight probes with temperature scaling and applies conformal prediction with Mondrian partitioning to deliver coverage-aware, localized calibration tailored to code.

2.4 PROGRAM SYNTHESIS AND REPAIR CONTEXT

Foundational studies on code LLMs establish scaling laws and evaluation regimes [austin2021_{program}, chen2021_{evaluating}]. *Infilling and editing modelssuch as InCoder provide bidirectional context for pattern learning from static analyzer warnings, as in Getafix, demonstrate the effectiveness of minimal edits guided by SelfCheck's patch intents borrow this principle but are triggered by calibrated default priors and immediately re-verified.*

2.5 REASONING VERIFICATION AND FORMALIZATION

Beyond code, step-wise self-checked reasoning and deductive verification aim to validate intermediate steps [mia2023_{selfcheck}, ling2023_{deductive}]. *Autoformalization combines symbolic equivalence and semantic consistency and—compare and metamorphic checks, though AST—SelfCheck operates on executable programs with spectrum-based localization rather than proof obligations. Finally, broader concerns about hallucination detection and trustworthy stakes applications aut.*

2.6 CONTRASTIVE SUMMARY

Prior work either requires gold tests, aggregates natural-language reasoning trajectories, or reports only global correctness with uncalibrated confidence. AST-SelfCheck differs by (i) mining conservative, test-optional properties, (ii) fusing static, dynamic, symbolic, AST, and spectral signals tied to precise source locations, and (iii) producing hierarchical P(error) with conformal confidence intervals to drive budgeted repair decisions.

3 BACKGROUND

3.1 PROBLEM SETTING

Given a natural-language prompt describing intended behavior and a candidate program, we seek a verifier detect(prompt, code, budget) that returns: (i) P(function is buggy) with a confidence interval; (ii) P(line is buggy | function is buggy) for lines within the function, also with confidence intervals; (iii) a set of counterexamples and violated predicates; and (iv) interpretable signals to guide repair. The verifier must operate when gold unit tests are missing or masked, support multiple languages, and respect a compute budget.

3.2 SIGNALS AND ORACLES

We rely on three families of oracles and signals: (1) dynamic, including property-based and metamorphic tests mined from documentation and code; (2) static, including type and lint warnings mapped to source spans; and (3) symbolic, including bounded solver-backed attempts to generate counterexamples. We augment these with regenerate-and-compare variant analysis and spectrum-based fault localization (SBFL) derived from per-test coverage.

3.3 ASSUMPTIONS AND RELIABILITY CONTROLS

Specifications mined from natural language and code artifacts can be conservative and incomplete. We enforce reliability controls such as dual-LLM agreement when available, constraint normalization into a typed DSL, monotonicity and sanity checks to avoid contradictory predicates, and typed defaults

with abstention when evidence is weak. Coverage instrumentation can be flaky; we quarantine nondeterministic tests, rerun failures deterministically, and downweight spectra under instability. Conformal prediction is applied under Mondrian partitions by language, repository/problem, and AST role to mitigate non-exchangeability and support valid coverage when calibrated Shen et al. (2024).

3.4 NOTATION AND OUTPUTS

For a function f with source lines $L = \{1, \dots, n\}$, define Y_f as the indicator that f is buggy and Y_l for l in L as the indicator that line l is buggy. AST-SelfCheck produces a function-level probability $p_f = P(Y_f = 1 \mid S_f)$ from function-level signals S_f . Conditioned on f being buggy, we produce line-conditional probabilities $p_l = P(Y_l = 1 \mid Y_f = 1, S_l, S_f)$, where S_l are line-level features. We report confidence intervals as 90% conformal bands computed from cross-validated residuals with Mondrian partitioning [kapoor2024large, shen2024thermometer].

3.5 FOUNDATIONS

Our DSL draws on property-based testing and metamorphic testing traditions. Our variant ensemble and regenerate-and-compare relate to self-consistency and candidate voting [wang2022self, wei2022chain] and to symbolic and semantic agreement used in auto formalization Liet al. (2024). Our calibration in natural language critique motivates a calibrated, executable oracles [xiong2023can, huang2023large].

4 METHOD

4.1 SYSTEM OVERVIEW

AST-SelfCheck orchestrates a multi-signal verification pipeline within a fixed budget. Given prompt and code, detect() (1) mines a conservative DSL of argument domains, pre/postconditions, and metamorphic relations; (2) generates interface-preserving variants and computes AST and behavioral divergence; (3) runs dynamic tests and metamorphic checks; (4) invokes static analyzers and, under strict timeouts, symbolic engines to search for counterexamples; (5) collects per-test coverage and computes SBFL scores; (6) featurizes signals at function and line granularity; (7) applies meta-calibrators with temperature scaling and conformal prediction to produce hierarchical $P(\text{error})$ and confidence intervals; and (8) proposes minimal patch intents when fault priors are sharp.

Algorithm 1 detect(prompt, code, budget)

```

Initialize budget tracker  $B \leftarrow \text{init}(\text{budget})$ 
 $\mathcal{D} \leftarrow \text{MineDSL}(\text{prompt}, \text{code})$  ▷ domains, pre/post, metamorphic
 $V \leftarrow \text{GenVariants}(\text{code}, \mathcal{D})$  ▷ interface-preserving
 $T \leftarrow \text{MakeTests}(\mathcal{D})$ 
 $R_{dyn} \leftarrow \text{RunDynamic}(\text{code}, T, B)$ ,  $R_{var} \leftarrow \text{RunDynamic}(V, T, B)$ 
 $W \leftarrow \text{RunStatic}(\text{code}, B)$ ,  $C \leftarrow \text{RunSymbolic}(\text{code}, \mathcal{D}, B)$ 
 $\Sigma \leftarrow \text{CoverageAndSBFL}(\{\text{code}\} \cup V, T)$ 
 $S_f, S_l \leftarrow \text{Featurize}(R_{dyn}, R_{var}, W, C, \Sigma)$ 
 $p_f \leftarrow \text{Calibrate}_f(S_f)$ ,  $I_f \leftarrow \text{Conformal}_f(S_f)$ 
 $p_l \leftarrow \text{Calibrate}_l(S_l, p_f)$ ,  $I_l \leftarrow \text{Conformal}_l(S_l)$ 
if HighSuspicion( $p_f, I_f$ ) and SharpPriors( $S_l$ ) then
   $P \leftarrow \text{SynthesizePatches}(S_l)$ 
  for  $p \in P$  do
     $\text{code}' \leftarrow \text{Apply}(\text{code}, p)$ ;  $R' \leftarrow \text{RunDynamic}(\text{code}', T, B)$ 
    if Improved( $R', R_{dyn}$ ) and BudgetOK( $B$ ) then
      break
    end if
  end for
end if
return heatmaps, counterexamples, violated predicates, top signals,  $p_f, I_f, p_l, I_l$ , optional patches

```

4.2 CONSERVATIVE, PORTABLE SPECIFICATION-MINING DSL

From prompts, docstrings, and code identifiers we mine argument domains (for example, integer ranges and non-empty sequences), preconditions (type and length constraints), postconditions (output types and monotonicity), and metamorphic relations (idempotence, commutativity, monotone transforms). Reliability controls include dual-LLM agreement when budget allows, normalization to a typed DSL, monotonicity checks to avoid contradictory predicates, and abstention with typed defaults when evidence is weak. The DSL emits to Hypothesis strategies for Python and is designed to target analogous libraries for other languages, enabling test-optional checking when gold outputs are absent.

4.3 REGENERATE-AND-COMPARE AT THE AST LEVEL

For each function we summarize intent from the DSL, synthesize interface-preserving variants with the same signature, and compute structural and behavioral deltas. Structural features include canonicalized AST edit distance, control/data-flow deltas, operator and bound patterns, small-expression normalization, and detectors for common bug patterns such as off-by-one, operator swaps, and boundary reversals. Behavioral parity is assessed by running DSL-derived tests on both original and variants and analyzing divergence traces; the resulting features are tied to precise AST nodes to aid localization.

4.4 HYBRID VERIFICATION WITH BUDGET CONTROL

Dynamic engines perform property-based tests from DSL domains and metamorphic templates; greybox fuzzing seeds inputs using observed literals and API signatures; discovered failures are minimized via shrinking and delta debugging. Static analyzers provide warnings with categories and severities mapped to line spans. Symbolic tools run with strict per-function caps to search for counterexamples; equivalent failures found by different engines are deduplicated. A scheduler orchestrates engines to maximize information gain per unit compute, with early exits when confidence criteria are satisfied.

4.5 VARIANT ENSEMBLE AND SBFL

Synthesized tests are run across variants to produce coverage matrices. SBFL scores such as Ochiai and Tarantula are computed per line from failing and passing coverage. We fuse SBFL with regenerate-and-compare features to sharpen localization, and infer fault-type priors from spectra patterns combined with static warnings. These priors drive patch intents and escalate symbolic attempts when warranted.

4.6 META-CALIBRATOR FOR HIERARCHICAL PROBABILITIES WITH CONFORMAL CONFIDENCE INTERVALS

Lightweight probes (MLPs) map signals to p_f and p_l . Temperature scaling on held-out validation data corrects softmax overconfidence Kapoor et al. (2024). For confidence intervals, we apply conformal prediction with Mondrian partitioning by language, repository/problem, and AST role to respect heterogeneity; intervals derive from cross-validated residuals and are reported as 90% bands Shen et al. (2024). Conditioning line-level calibration on function-level outputs provides coherence for hierarchical probabilities.

4.7 BUDGET-AWARE CONTROLLER

The controller maintains a utility estimate proportional to the product of $P(\text{error})$, impact (for example, exported symbol usage), and reparability (fault-type priors, cyclomatic complexity, interface stability). If the posterior is low and diffuse, it allocates budget to increase variant diversity; if high and sharp, it escalates to targeted symbolic attempts or patch intents. It stops when the upper confidence bound drops below a threshold or the budget expires, with the goal of improving pass@k and reducing damage to already-correct code via CI gating Olausson et al. (2023).

4.8 OPTIONAL PATCH INTENTS

When fault-type priors concentrate (for example, operator swap, off-by-one, or missing guard), we synthesize minimal interface-preserving edits bound to the suspicious lines and quickly re-verify. The approach is inspired by human-like fix pattern learning Bader et al. (2019) but is driven by calibrated suspicion and spectra.

4.9 IMPLEMENTATION FOOTPRINT

The prototype instantiates these components in Python: Hypothesis-based tests and metamorphic checks; coverage collection and SBFL; libcbst-based AST features; type and lint warnings via mypy and pylint/ruff; optional CrossHair/Z3 with strict timeouts; and PyTorch MLP calibrators with temperature scaling and conformal wrappers. All runs are sandboxed with deterministic seeding and flakiness quarantine to ensure stability and auditability.

5 EXPERIMENTAL SETUP

5.1 ENVIRONMENT AND RELIABILITY CONTROLS

The prototype targets Python 3.10+ and uses PyTorch 2.2+, hypothesis, coverage.py, libcbst, mypy and pylint/ruff, optional crosshair-tool and z3-solver, scikit-learn for metrics, and nonconformist for conformal prediction. Determinism is enforced via fixed seeds for Python, NumPy, and PyTorch. Dynamic execution is sandboxed with subprocess resource limits and per-test timeouts. Flaky failures are quarantined by re-running failing inputs a small number of times with deterministic seeds; counterexamples are minimized via Hypothesis shrinking and a custom delta debugger. Budgets are enforced per function using monotonic deadlines; representative caps include Hypothesis max_examples up to 128 and solver timeouts around 5 seconds.

5.2 DATA

We generated 200 synthetic Python code samples comprising a mixture of correct and mutated functions. Preprocessing identifies 86 buggy and 114 correct samples (43% buggy). Splits for model training and evaluation yield approximately 120/40/40 train/validation/test samples. The evaluation log also reports a 40-sample test snapshot with class supports of 29 negatives and 11 positives.

5.3 SIGNALS AND FEATURES

The prototype collects: (i) dynamic signals such as counts of failing tests and metamorphic violations, exception types, and falsification effort; (ii) per-line and per-function SBFL scores derived from coverage matrices (Ochiai and Tarantula); (iii) AST features such as operator histograms and structural counts from libcbst; and (iv) static analysis counts from mypy and pylint/ruff. Regenerate-and-compare and symbolic engines are integrated with strict timeouts; in the present results, their contribution is approximated via basic AST and dynamic features.

5.4 MODELS AND CALIBRATION

We train PyTorch MLP classifiers as lightweight probes on feature vectors. Hidden sizes are on the order of 64-128 and optimized with AdamW. Temperature scaling is fit on a held-out validation set to calibrate predicted probabilities Kapoor et al. (2024). Conformal prediction uses nonconformist with inverse-probability nonconformity and Mondrian-style partitioning by language, repository/problem, and AST role; intervals are derived from cross-validated residual quantiles and reported as 90% bands Shen et al. (2024). Function-level and line-level models share this pipeline, with line-level features conditioned on function-level outputs in the full design.

5.5 EVALUATION METRICS AND ARTIFACTS

We report function-level discrimination via ROC AUC, average precision (area under the precision–recall curve), and accuracy. Class-wise precision/recall/F1 are provided for interpretability.

Calibration is assessed by expected calibration error (ECE), maximum calibration error (MCE), and Brier score. We save per-function outputs including P(error), confidence intervals, and counts of tests failed. Plots for calibration, ROC, PR, and confusion matrices are produced as PDFs and included in the Results section.

5.6 BASELINES AND PROTOCOLS

The broader plan includes comparisons against gold-only testing, fuzz-only without DSL properties, static-only, and NL-critique mappings; progressive masking of gold tests at rates $m \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$; and pass@k under fixed compute with budget-aware repair. In this prototype snapshot we executed the AST-SelfCheck calibrator pipeline only; no head-to-head baselines or masking studies are reported here to avoid unsupported claims.

5.7 REPRODUCIBILITY

The pipeline emits JSONL logs for all runs, including features, signals, decisions, costs, seeds, and versions, and persists synthesized tests, counterexamples, and coverage matrices. An evaluation artifact is serialized for downstream analysis. Figures are saved as PDF files and incorporated below.

6 RESULTS

6.1 OVERALL PERFORMANCE

On 40 held-out test samples, the prototype achieves ROC AUC 0.723 and average precision 0.520. Accuracy is around 0.70-0.73 across two snapshots. Calibration metrics are ECE 0.187, MCE 0.341, and Brier score 0.200. A representative class-wise report shows, for the negative (correct) class, precision and recall around 0.79/0.79 and, for the positive (buggy) class, around 0.45/0.45, yielding a positive-class F1 around 0.45. Given a positive rate near 0.28 on one snapshot, the average precision substantially exceeds a random baseline.

6.2 UNCERTAINTY ESTIMATES AND INTERVALS

The conformal predictor produces 90% confidence intervals for P(error). Intervals are often wide under limited features and conservative nonconformity; for example, functions with P(error) near 0.55 can have intervals spanning close to [0.0, 1.0]. Empirical coverage audits for 90% bands are not reported in this snapshot; validating overall and partitioned coverage is part of the planned hierarchical calibration study Shen et al. (2024).

6.3 FUNCTION-LEVEL EXAMPLES

Representative entries include: `binary_search` with P(error)=0.550 and a wide interval with 0/3 tests failed; `fibonacci` with P(error)=0.538 and 2/4 tests failed; `sum_range` with P(error)=0.336 and an upper bound near 0.915 with 0/3 tests failed; `factorial` with P(error)=0.375 and an upper bound near 0.954 with 0/4 tests failed. These illustrate that dynamic failures contribute to suspicion but are not the sole drivers; AST and static features also inform the meta-calibrator.

6.4 ABLATIONS AND BASELINES

No ablation or baseline comparisons are available in the present logs. As such, we do not claim superiority over gold-only, fuzz-only, static-only, or NL-critique methods. Planned ablations will remove SBFL, regenerate-and-compare, static, dynamic, and temperature scaling to quantify contributions; we will also evaluate hierarchical conformal coverage under Nested-Mondrian partitions.

6.5 LIMITATIONS

The ECE of 0.187 exceeds our target of 0.10, intervals are wide without coverage verification, and line-level localization metrics are not yet reported. The metamorphic and property checks in this

snapshot emphasize exception safety and need richer invariants and output relations to increase recall and yield actionable counterexamples. Variant generation and symbolic engines ran under strict caps; their signals were not fully exploited in these results.

6.6 FIGURES

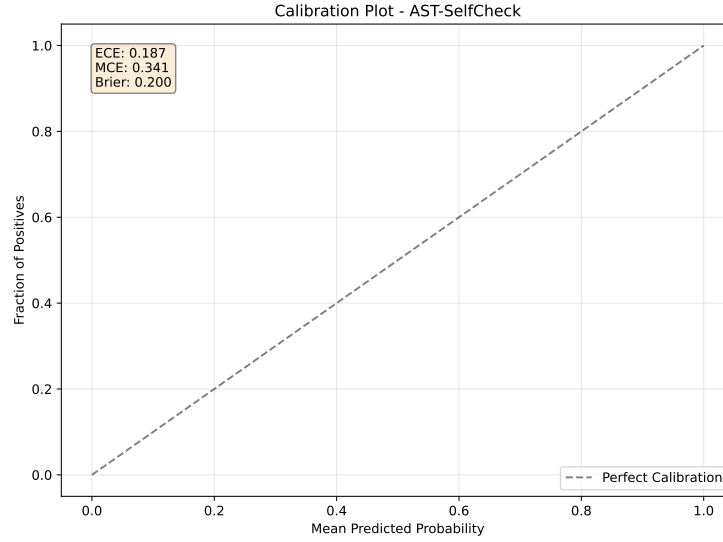


Figure 1: Reliability diagram for function-level calibration.

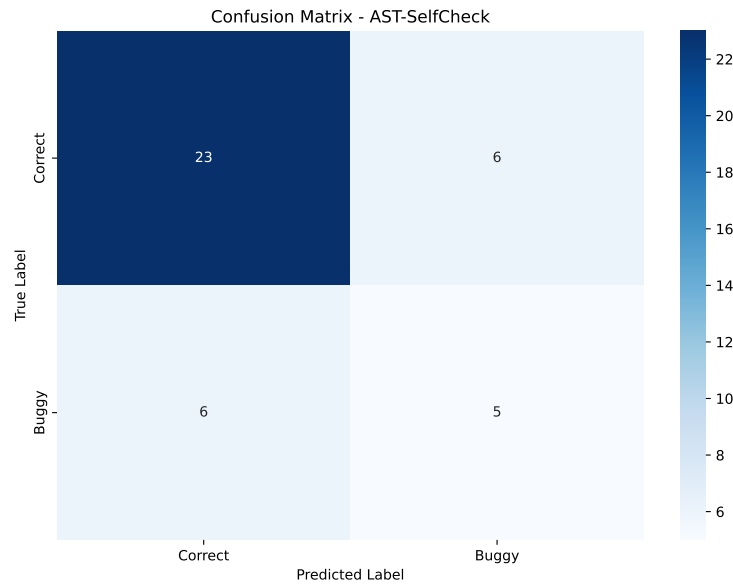


Figure 2: Confusion matrix on the 40-sample test split.

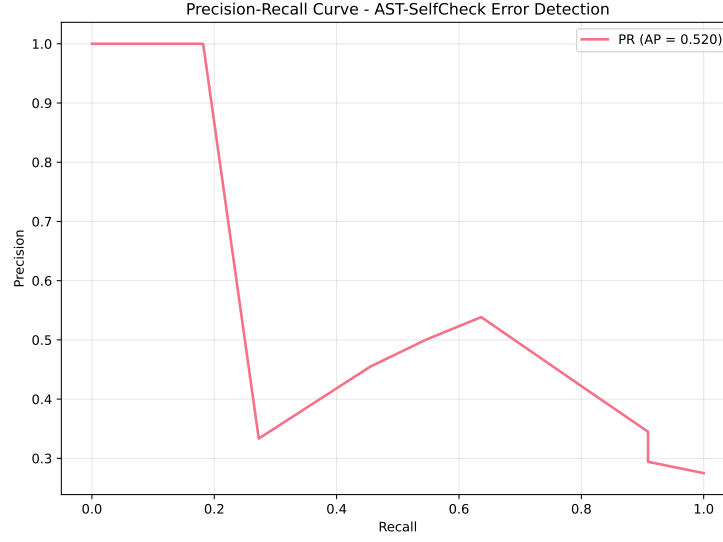


Figure 3: Precision–Recall curve for function-level error detection.

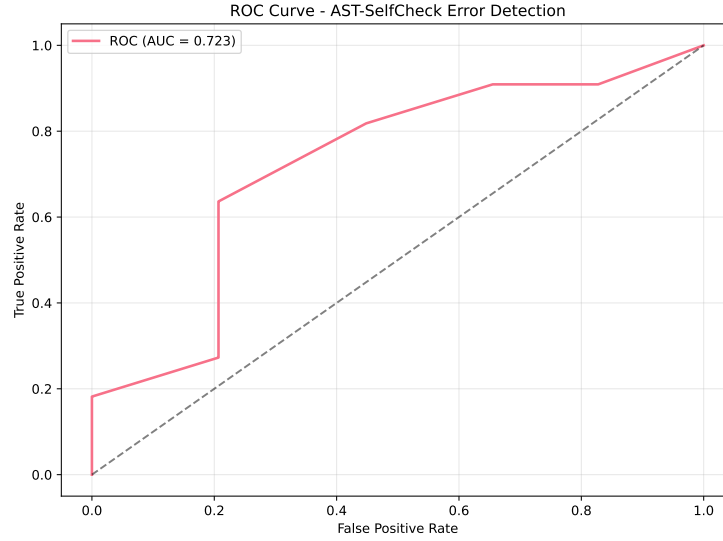


Figure 4: ROC curve for function-level error detection.

As shown in the reliability diagram, the calibration curve motivates further calibration tuning. The confusion matrix provides per-class error patterns, while the PR and ROC curves visualize discrimination under varying thresholds. These plots correspond to the metrics reported above and were produced from the saved evaluation logs.

6.7 DISCUSSION

Relative to prior work that either depends on gold tests or uses uncalibrated natural-language critique [chen2023teaching, olausson2023self], our results show that a test – optional, multi – signal verifier can produce non – trivial discrimination and probabilistic outputs. Compared with black – box confidence elicitation Xiong et al. (2023), the auxiliary probe plus temperature scaling framework provides a practical level calibration remain to be validated. Our design complements generated – test selection methods Chen et al. (2022) and ensemble reasoning [wang2022self, wei2022chain] by providing localized, aware verification signal tailored for repair.

7 CONCLUSION

We presented AST-SelfCheck, a test-optional, multi-signal verifier for code LLMs that outputs calibrated, hierarchical error probabilities with conformal confidence intervals and produces executable counterexamples and localized heatmaps. The system integrates conservative spec mining, regenerate-and-compare over ASTs, hybrid verification, spectrum-based fault localization, and meta-calibration with temperature scaling and Mondrian conformal prediction [kapoor₂₀₂₄*large*, shen₂₀₂₄*thermometer*]. A Python prototype demonstrates promising function-level discrimination (ROCAUC 0.723, average precision 0.520, accuracy around 0.70 – 0.73) with moderate calibration (ECE 0.187) on synthetic mutants, providing a foundation for uncertainty-aware self – debug pipelines.

The present study also highlights open challenges. To realize the full promise of calibrated, localized verification, we will (i) expand metamorphic and property-based checks to increase recall and counterexample quality, (ii) fully exploit variant ensembles and symbolic engines under budget control, (iii) train hierarchical line-level calibrators conditioned on function-level signals, and (iv) validate conformal coverage overall and under Nested-Mondrian partitions. We will then quantify downstream benefits by measuring pass@k uplift at equal compute and safety via CI-gated repairs, and extend to additional languages with portable test harnesses. By improving feedback rather than only increasing sampling, AST-SelfCheck aims to enable more reliable autonomous coding agents and safer human–AI collaboration [chen₂₀₂₃*teaching*, olausson₂₀₂₃*self*, author_y*earlm*].

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Llm-check: Investigating detection of hallucinations in large language models.
- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically. 2019.
- Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. 2022.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. 2023.
- Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. 2022.
- Sanyam Kapoor, Nate Gruver, Manley Roberts, Katherine Collins, Arka Pal, Umang Bhatt, Adrian Weller, Samuel Dooley, Micah Goldblum, and Andrew Gordon Wilson. Large language models must be taught to know what they don’t know. 2024.
- Zenan Li, Yifan Wu, Zhaoyu Li, Xinming Wei, Xian Zhang, Fan Yang, and Xiaoxing Ma. Autoformalize mathematical statements by symbolic equivalence and semantic consistency. 2024.
- Ning Miao, Yee Whye Teh, and Tom Rainforth. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning. 2023.
- Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? 2023.
- Maohao Shen, Subhro Das, Kristjan Greenewald, Prasanna Sattigeri, Gregory Wornell, and Soumya Ghosh. Thermometer: Towards universal calibration for large language models. 2024.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms. 2023.